

2110531 - Data Science and Data Engineering Tools

MLOPS

Asst.Prof. Natawut Nupairoj, Ph.D.
Department of Computer Engineering
Chulalongkorn University
natawut.n@chula.ac.th

CHULA ENGINEERING
Foundation toward Innovation

Outline

1

Why more tools?

- Questions notebooks can't answer
- Data, models and runs: three histories
- Where each tool fits

2

MLflow: track and register

- Runs, params, metrics, models
- Link runs to DuckLake snapshots
- Model registry and aliases

3

Prefect: orchestrate

- Tasks, flows and states
- Retries need idempotent steps
- Schedules and deployments

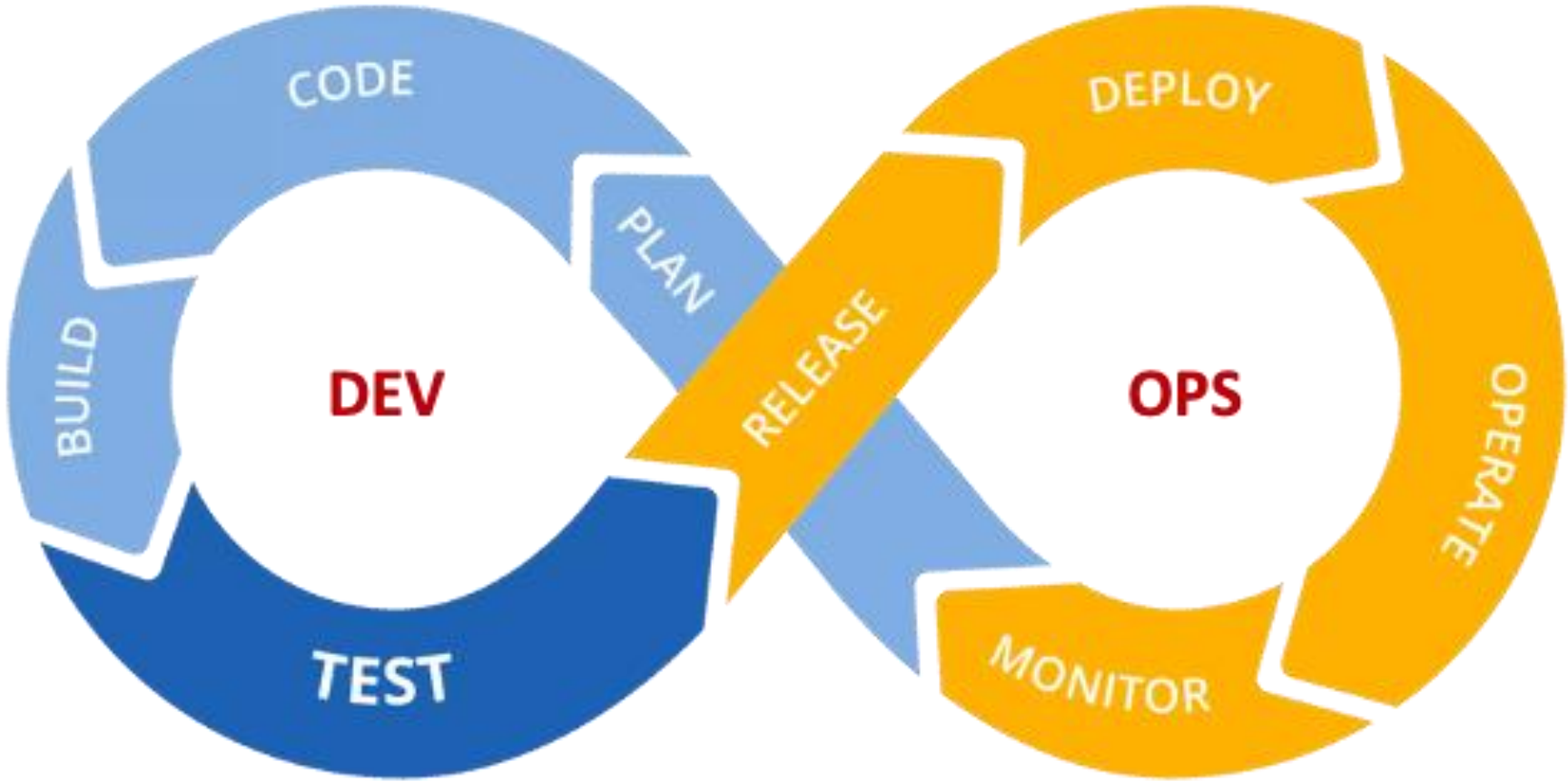
4

Putting it together

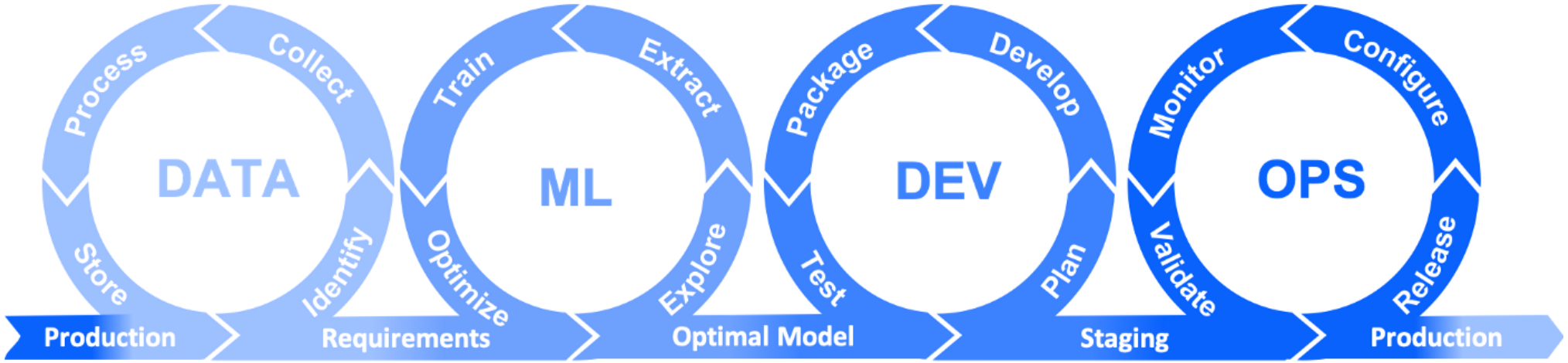
- One nightly flow, end to end
- Airflow, Dagster and others
- Wrap-up and exercises

Builds on: the phase notebooks (pyducklake) and ml/tutorial_spark_ml.ipynb · data: synthetic MetroRide trips

The Infinite Lifecycle



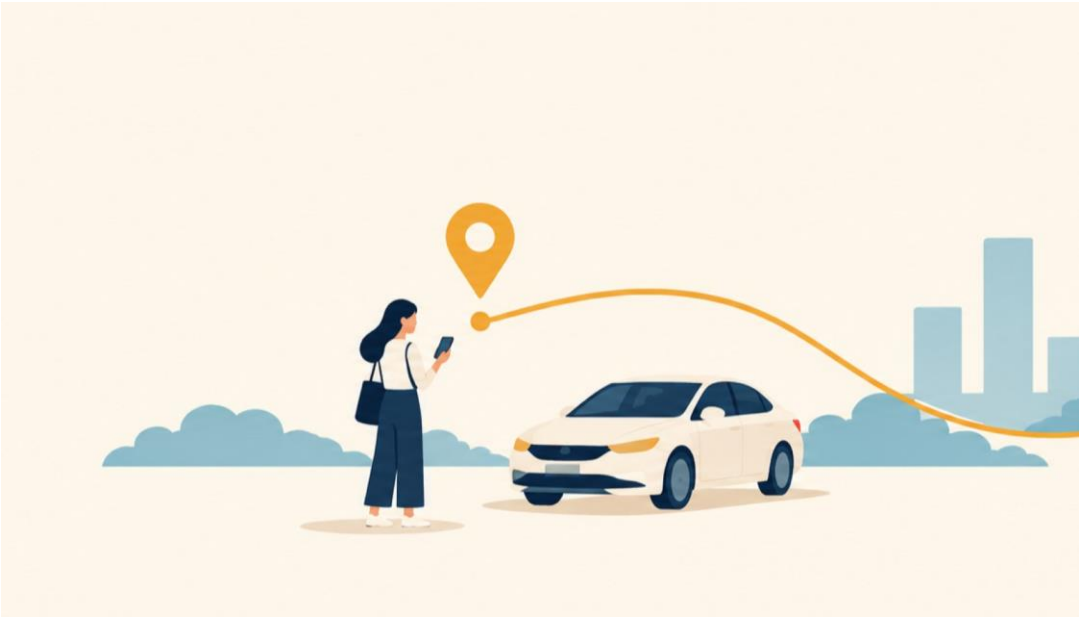
Put ML into production, these cycles become more complicated



Source: https://medium.com/@bhandaripallavi.98/mlops-ease-collaboration-between-data-scientists-and-it-ops-e7e3ccac9eed

Part 1

From notebooks to a pipeline



Questions the notebooks cannot answer

1	*"What did silver.trip_context look like when we trained the model?"*	→	**DuckLake snapshots** data versions (done in phases 1–3)
2	*"Which features and data gave MAE 1.90? Can we get that model back?"*	→	**MLflow Tracking** runs: params, metrics, models
3	*"Which model version is the passenger app using today?"*	→	**MLflow Model Registry** versions and aliases
4	*"Who loads tonight's CSV at 02:00, and what if it crashes halfway?"*	→	**Prefect** scheduled, retried, logged runs

Three tools, three kinds of history

DuckLake

versions the data

Unit: snapshot: every commit to the catalog

"What did the table look like then?"

```
silver.scan()
  .with_snapshot(24)
```

MLflow

versions experiments and models

Unit: run, registered model version

"Which run is best? Which model is live?"

```
models:/metroride-arrival-wait
  @champion
```

Prefect

versions pipeline executions

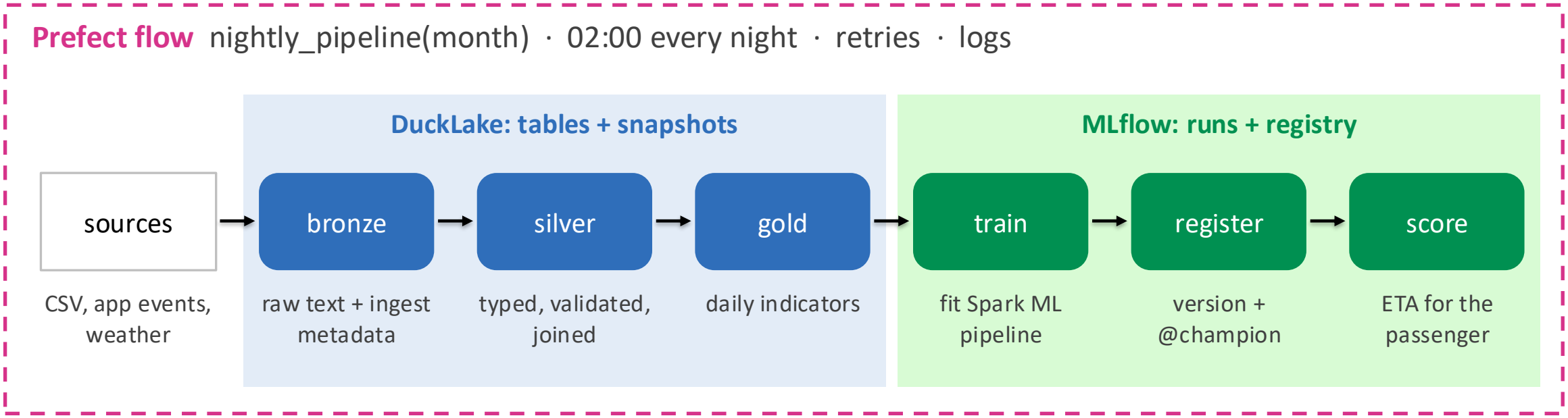
Unit: flow run and task runs, each with a state

"What ran last night? Did it succeed?"

```
nightly_pipeline(
  month="2026-09")
```

The glue: each MLflow run stores the DuckLake snapshot_id it trained on, and one Prefect flow run does the load, the transforms and the training.

Where each tool fits in the MetroRide pipeline



Existing

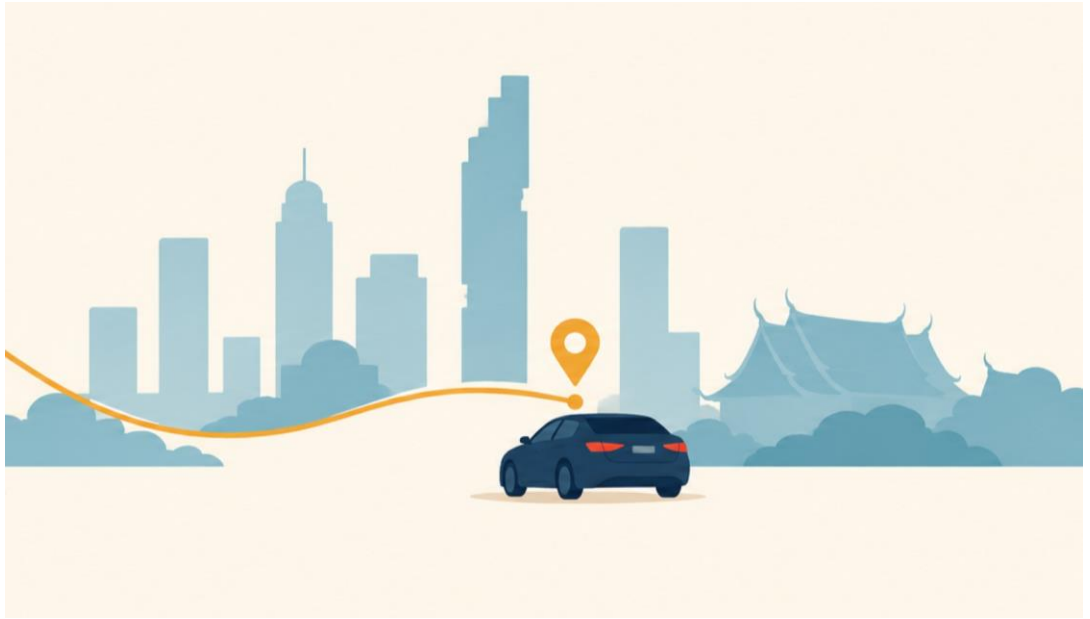
Data transformation notebooks: bronze → silver → gold, run by hand, one cell at a time. Spark ML notebook: train and score.

New today

MLflow records every training run and serves the chosen model. Prefect runs every step, in order, on a schedule.

Part 2

MLflow: track experiments and models



What is in MLflow?

Tracking

Record params, metrics, tags and files for every run

`mlflow.start_run, log_params, log_metric, set_tag`

Models

Save a model with its flavor and Python environment

`mlflow.spark.log_model, mlflow.sklearn, mlflow.pyfunc`

Model Registry

Named models with versions and aliases

`register_model, set_registered_model_alias`

UI and search

Compare runs side by side; query them in code

`mlflow server, mlflow ui, mlflow.search_runs`

Autologging

One call logs params, metrics and the model

`mlflow.pyspark.ml.autolog(), mlflow.sklearn.autolog()`

Evaluation and GenAI

Evaluate models; trace and test LLM apps

`mlflow.evaluate, mlflow.genai, @mlflow.trace`

Green = used today · open source (Apache 2.0) · works with Spark ML, scikit-learn, PyTorch, …

CHULA ENGINEERING
Foundation toward Innovation

Tracking: an experiment holds runs

Experiment `metroride-arrival-wait`

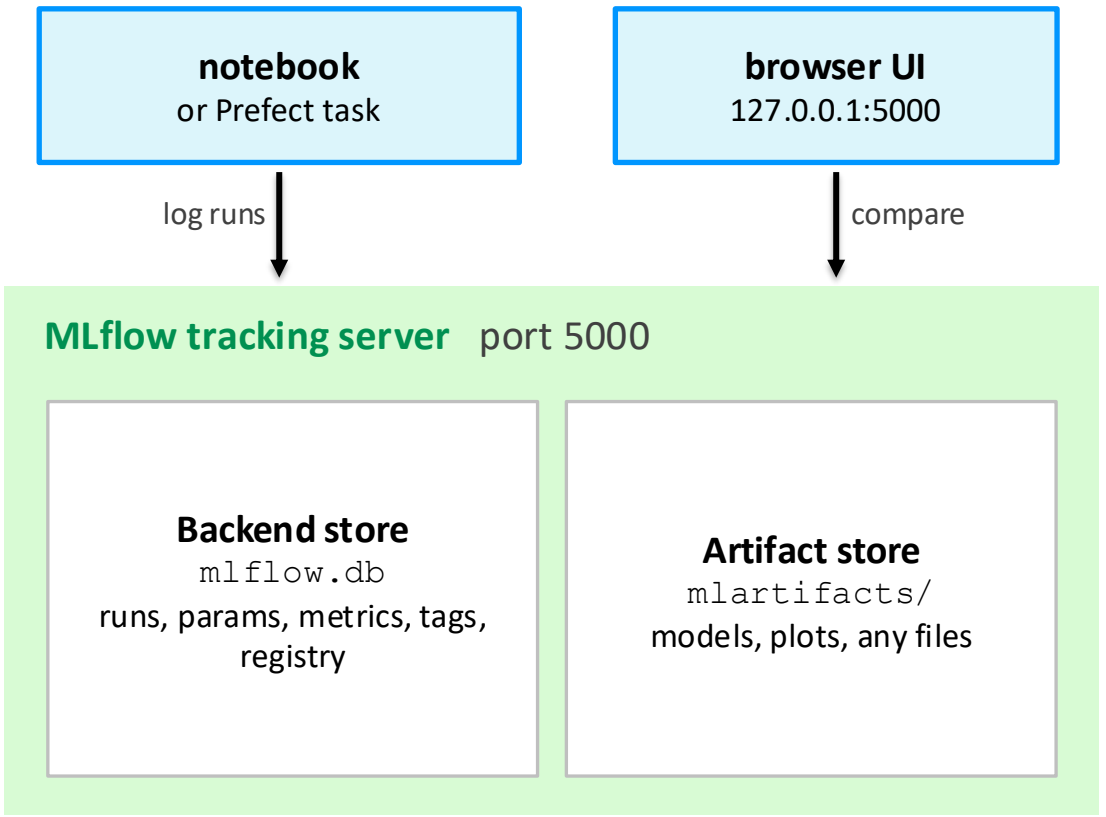
one question, one test set (September)

run	**baseline: training mean**	`mae 1.97`
run	**zone + slot + weekday**	`mae 2.03`
run	**+ accept delay**	`mae 1.90`
run	**LEAKY: + pickup wait**	`mae 0.00`

One run records

Kind	Example from the tutorial	Call
params	numeric = ['accept_delay_minutes'] split = train Jun–Aug	`log_params`
metrics	mae 1.90, rmse, r2	`log_metric`
tags	ducklake.snapshot_id = 24	`set_tag`
artifacts	model/ (the Spark PipelineModel), plots, files	`log_model`, `log_artifact`
automatic	start and end time, user, source file, git commit	`(none)`

Set up: one tracking server for the class



Terminal, once

```
pip install mlflow
mlflow server --port 5000 \
    --backend-store-uri sqlite:///mlflow.db
```

Notebook, at the top

```
import mlflow
mlflow.set_tracking_uri("http://127.0.0.1:5000")
mlflow.set_experiment("metroride-arrival-wait")
```

No server? set_tracking_uri("sqlite:///mlflow.db") writes to a local file; run mlflow ui in the same folder to browse it.

Green = lines you add to the existing notebook

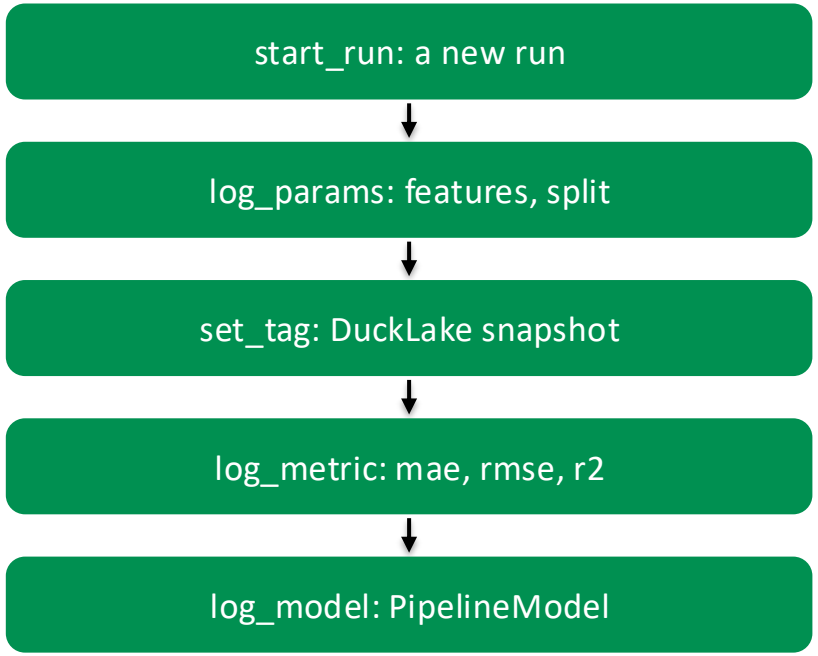
CHULA ENGINEERING
Foundation toward Innovation

Step 1 · Wrap fit_and_score() in a run

```
def fit_and_score(name, categorical, numeric,
                  train_df=train, test_df=test):
    with mlflow.start_run(run_name=name):
        mlflow.log_params({"categorical": categorical,
                           "numeric": numeric,
                           "split": "train Jun-Aug, test Sep"})
        mlflow.set_tag("ducklake.snapshot_id", SNAPSHOT_ID)
        stages = build_stages(categorical, numeric)
        model = Pipeline(stages=stages).fit(train_df)
        pred = model.transform(test_df)
        for m in ["mae", "rmse", "r2"]:
            ev = RegressionEvaluator(labelCol=LABEL,
                                     metricName=m)
            mlflow.log_metric(m, ev.evaluate(pred))
        mlflow.spark.log_model(model, artifact_path="model")
    return model

m2 = fit_and_score("+ accept delay",
                   ["service_zone", "time_slot", "weekday"],
                   ["accept_delay_minutes"])
```

What one call records



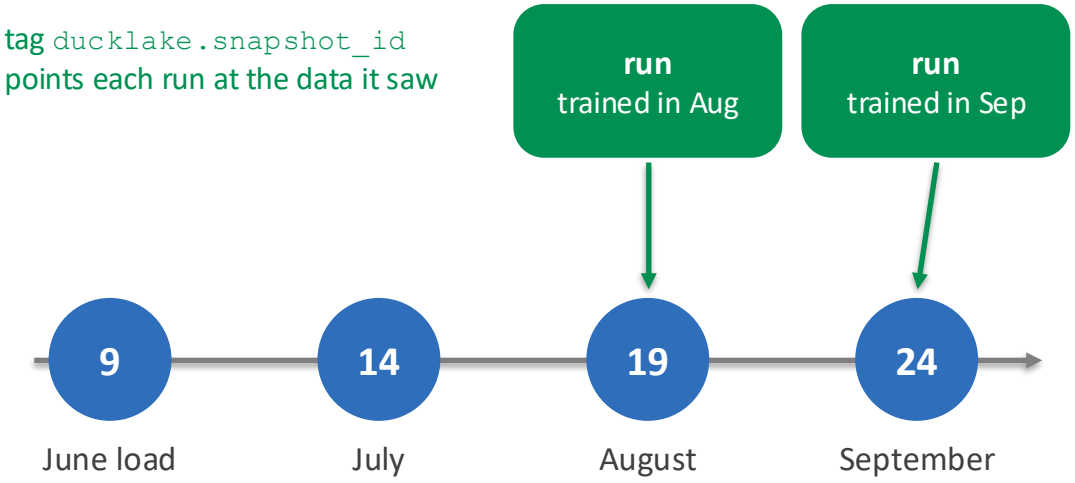
The Spark ML code is unchanged; MLflow only records what happened.

Green = lines you add · build_stages() = the StringIndexer, OneHotEncoder, VectorAssembler and LinearRegression stages from the tutorial

Step 2 · Link every run to a DuckLake snapshot

`silver.trip_context` snapshots

tag `ducklake.snapshot_id`
points each run at the data it saw



Reproducible = same code (git) + same params (MLflow) + same data (DuckLake snapshot).

When training

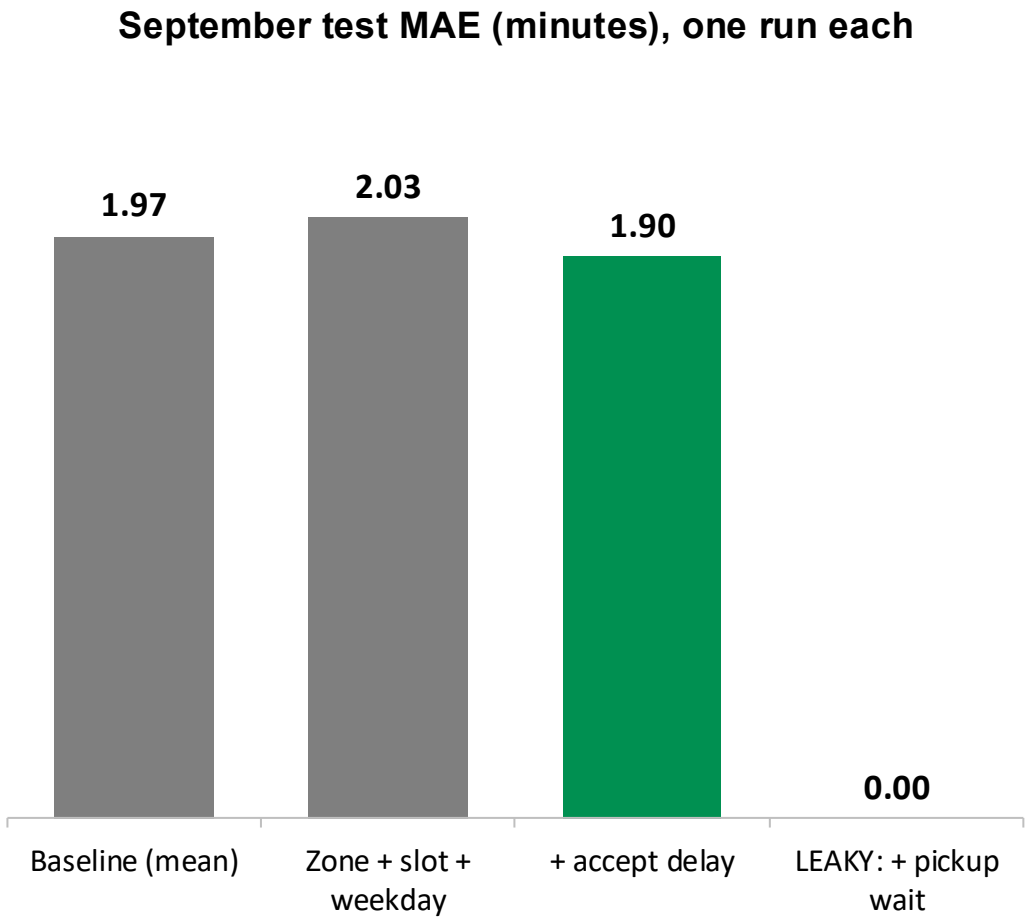
```
tc = catalog.load_table(("silver", "trip_context"))
SNAPSHOT_ID = tc.current_snapshot().snapshot_id
data = to_spark(tc, snapshot_id=SNAPSHOT_ID)
...
mlflow.set_tag("ducklake.snapshot_id", SNAPSHOT_ID)
```

Months later: get the exact training rows back

```
run = mlflow.get_run(run_id)
snap = int(run.data.tags["ducklake.snapshot_id"])
same_rows = to_spark(tc, snapshot_id=snap)
```

to_spark(table, snapshot_id) = the Spark ↔ DuckLake bridge from the PySpark tutorial · snapshot ids are examples

Step 3 · Compare runs: sort, filter, spot the leak



Find the best runs in code

```
runs = mlflow.search_runs(
    experiment_names=["metroride-arrival-wait"],
    order_by=["metrics.mae ASC"])
runs[["tags.mlflow.runName", "metrics.mae"]]
```

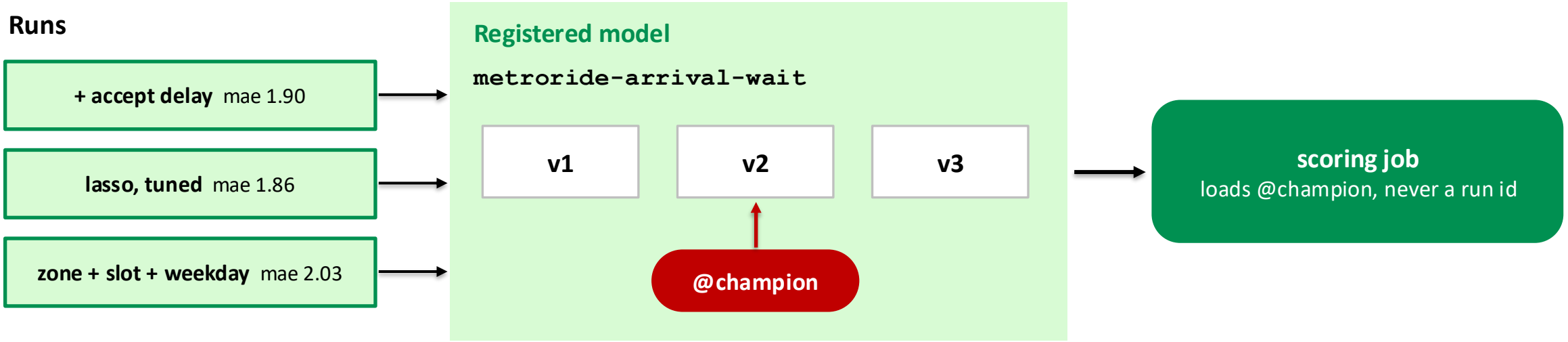
MAE 0.00 and R^2 1.0 is a red flag, not a win

The LEAKY run uses the passenger pickup wait, which is only known after the driver arrives.

Compare only within one experiment

The September-only rain model (MAE 1.24) uses another test set, so it goes in its own experiment.

Step 4 · Register the model, point an alias at it



Register and promote

```
info = mlflow.spark.log_model(model, artifact_path="model")
mv = mlflow.register_model(info.model_uri,
                           "metroride-arrival-wait")
MlflowClient().set_registered_model_alias(
    "metroride-arrival-wait", "champion", mv.version)
```

Load in the scoring job

```
champion = mlflow.spark.load_model(
    "models:/metroride-arrival-wait@champion")
champion.transform(new_trips)   # MAE 1.90 again
```

Promote = move the alias. Roll back = move it back. The scoring code never changes.

Part 3

Prefect: orchestrate the pipeline

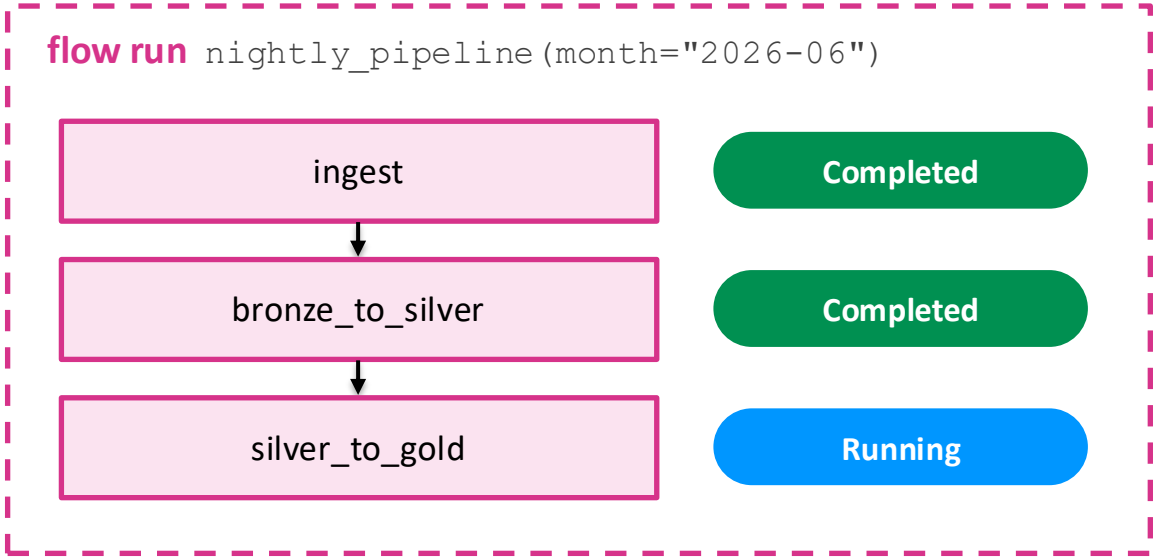


Why not cron and a notebook?

Need	cron + script	Orchestrator (Prefect, Airflow, Dagster, …)
Run steps in order, stop on failure	you write it	tasks inside a flow; a failed task fails the flow
Retry a flaky step	you write it	@task(retries=2, retry_delay_seconds=30)
See what ran, when, and why it failed	log files on one machine	UI with every flow run, task state and log
Re-run one month (backfill)	edit the script, run by hand	run the flow with month="2026-06"
Schedule	crontab, server time zone	cron schedule with time zone; pause, resume
Alert someone	–	notifications when a run fails

The phase notebooks already say "the scheduler re-runs the job": an orchestrator is that scheduler.

Prefect building blocks



Run states



Scheduled → Running → Completed, or Failed → AwaitingRetry → Running

Term	Meaning
`task`	a function Prefect tracks: state, retries, logs, timing
`flow`	a function that calls tasks; each call is one flow run
`state`	where a run is now; the UI shows the history
`parameters`	flow arguments, e.g. month, set per run
`deployment`	a flow + schedule + default parameters, stored on the server
`server / UI`	keeps runs and logs; http://127.0.0.1:4200

Step 1 · Decorate the functions you already have

```python
from prefect import flow, task
from prefect.cache_policies import NONE

@task(retries=2, retry_delay_seconds=30, cache_policy=NONE)
def ingest(month: str) -> None:
    ingest_month(month)                  # phase-2 notebook

@task(retries=2, retry_delay_seconds=30, cache_policy=NONE)
def bronze_to_silver() -> None:
    transform_bronze_to_silver()

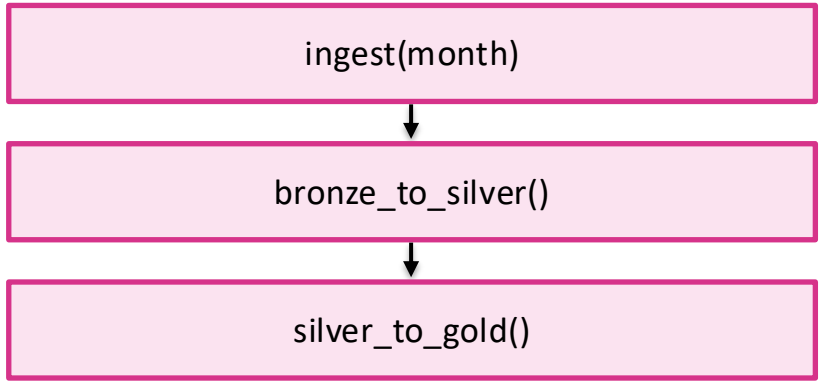
@task(cache_policy=NONE)
def silver_to_gold() -> None:
    transform_silver_to_gold()

@flow(name="metroride-nightly", log_prints=True)
def nightly_pipeline(month: str):
    ingest(month)                        # runs in order;
    bronze_to_silver()                   # a failure stops
    silver_to_gold()                     # the flow

nightly_pipeline("2026-06")              # one flow run
```

Green = lines you add · function bodies are the phase-2 notebook functions, unchanged

What you get



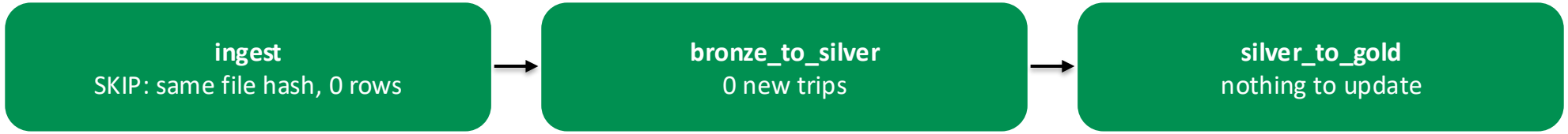
- Each call is a task run with a state, log and timing
- A failed task is retried twice, 30 s apart
- If it still fails, the flow stops: gold is never built from half-loaded silver

Step 2 · A crash, a retry, and no duplicate rows

Flow run 1
02:00, May file



Flow run 2
re-run by mistake



`bronze.trips_raw` rows

132	269	269
before (April)	after run 1	after run 2

Retries are only safe because every step is idempotent

- file-hash guard and trip_id row guard in bronze
- NOT EXISTS anti-join from bronze to silver
- one transaction per write: a crash inside it writes nothing

Step 3 · Schedule it with a deployment

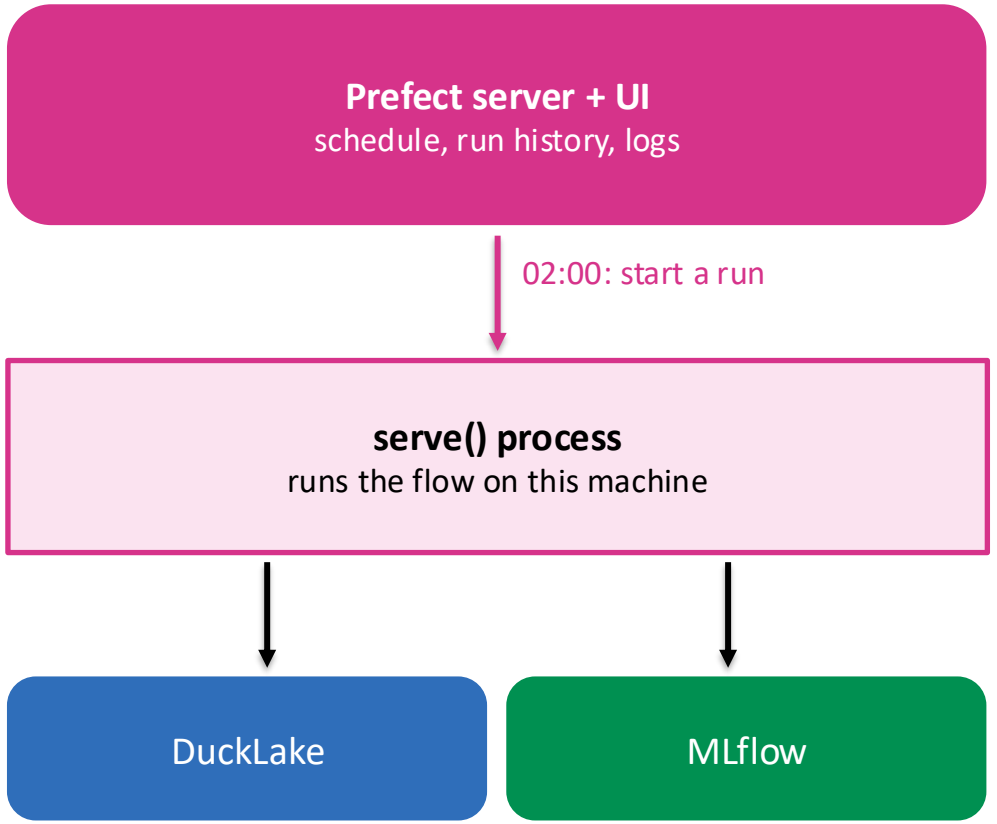
pipeline.py, at the bottom

```
from prefect.schedules import Cron

if __name__ == "__main__":
    nightly_pipeline.serve(
        name="nightly",
        schedule=Cron("0 2 * * *", timezone="Asia/Bangkok"),
        parameters={"month": "2026-09"})
```

Terminal

```
prefect server start      # UI: http://127.0.0.1:4200
python pipeline.py        # registers, then waits
prefect deployment run "metroride-nightly/nightly" \
    -p month=2026-06      # backfill June now
```



serve() = simplest deployment, one long-running process. Production: work pools and workers (Docker, Kubernetes, cloud).

Gotchas: DuckLake, Spark and Prefect together

1 One writer at a time

Our DuckLake catalog is one DuckDB file. Call write tasks one after another; don't .submit() or .map() them in parallel. For many writers, use a PostgreSQL catalog.

2 Pass names, not DataFrames

Tasks read and write the lakehouse themselves and return small values: row counts, snapshot ids, model URIs.

```
return info.model_uri, mae
```

3 Turn off input caching

Prefect can cache a task by hashing its inputs. A Catalog or SparkSession can't be hashed, and the lakehouse changes anyway.

```
@task(cache_policy=NONE)
```

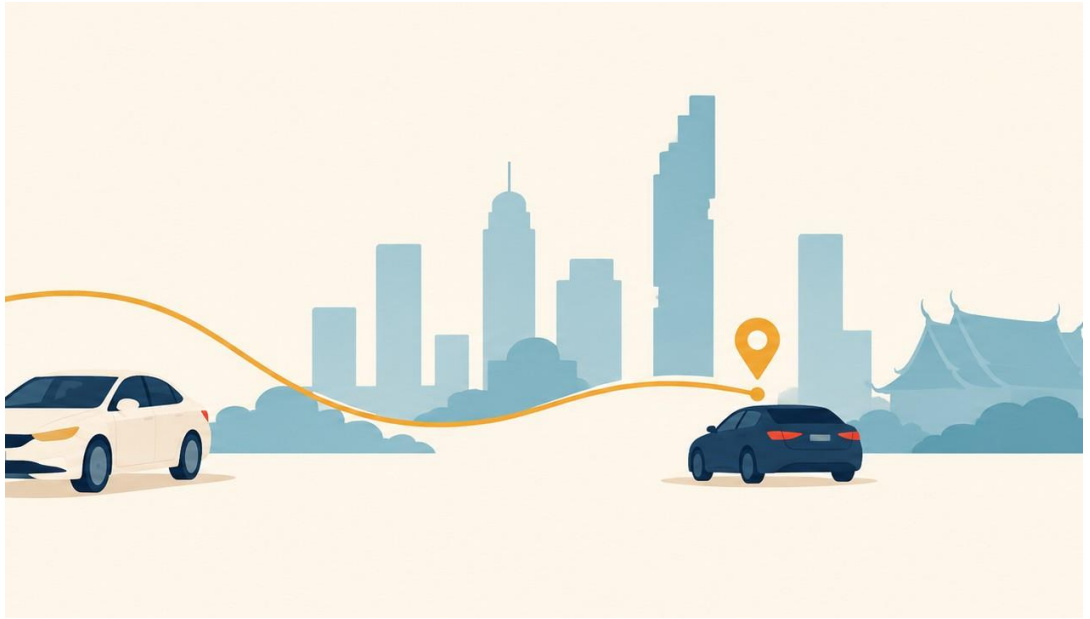
4 One SparkSession per flow run

Create it inside the flow, stop it at the end. Spark already runs in parallel inside one task.

```
spark = SparkSession.builder.getOrCreate()
```

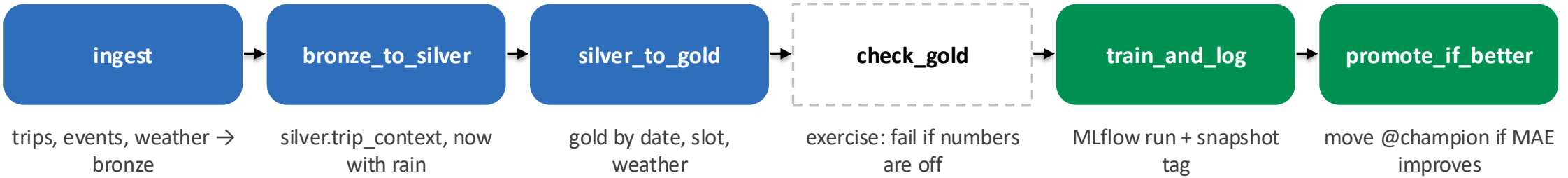
Part 4

Putting it together



September arrives: one flow run does it all

flow run `nightly_pipeline(month="2026-09")`



DuckLake

a new snapshot for every commit; gold rows split by weather class

MLflow

a run named after the snapshot; a new model version if it beats the champion

Prefect UI

one Completed flow run, six task runs, all logs; a red run if anything failed

The training and promotion tasks

```python
@task(cache_policy=NONE)
def train_and_log() -> tuple[str, float]:
    tc = catalog.load_table(("silver", "trip_context"))
    snap = tc.current_snapshot().snapshot_id
    with mlflow.start_run(run_name=f"nightly {snap}"):
        mlflow.set_tag("ducklake.snapshot_id", snap)
        model, mae = fit(to_spark(tc, snapshot_id=snap))
        mlflow.log_metric("mae", mae)
        info = mlflow.spark.log_model(
            model, artifact_path="model")
    return info.model_uri, mae

@flow(name="metroride-nightly", log_prints=True)
def nightly_pipeline(month: str):
    ingest(month)
    bronze_to_silver()
    silver_to_gold()
    uri, mae = train_and_log()
    print("promoted:", promote_if_better(uri, mae))
```

```python
@task
def promote_if_better(uri: str, mae: float) -> bool:
    client = MlflowClient()
    name = "metroride-arrival-wait"
    try:
        champ = client.get_model_version_by_alias(
            name, "champion")
        best = client.get_run(champ.run_id) \
                     .data.metrics["mae"]
    except MlflowException:       # no champion yet
        best = float("inf")
    if mae >= best:
        return False              # keep the champion
    mv = mlflow.register_model(uri, name)
    client.set_registered_model_alias(
        name, "champion", mv.version)
    return True
```

fit() = time split + Pipeline from the Spark ML tutorial, returns (model, test MAE) · comparing MAE across nights assumes the same test month

Same pipeline in three orchestrators

Prefect

plain Python: decorate functions, call them

```
@task(retries=2)
def ingest(month): ...

@task
def bronze_to_silver(): ...

@task
def silver_to_gold(): ...

@flow
def nightly_pipeline(month):
    ingest(month)
    bronze_to_silver()
    silver_to_gold()

nightly_pipeline.serve(
    cron="0 2 * * *")
```

Apache Airflow

a DAG of tasks; scheduler and workers

```
import pendulum
from airflow.sdk import dag, task

@dag(schedule="0 2 * * *",
     start_date=pendulum.datetime(
         2026, 4, 1,
         tz="Asia/Bangkok"),
     catchup=False)
def metroride_nightly():
    @task(retries=2)
    def ingest(): ...
    @task
    def bronze_to_silver(): ...
    @task
    def silver_to_gold(): ...
    (ingest() >> bronze_to_silver()
              >> silver_to_gold())

metroride_nightly()
```

Dagster

assets: declare tables and what they need

```
from dagster import asset

@asset
def trips_raw(): ...          # bronze

@asset(deps=[trips_raw])
def trips(): ...              # silver

@asset(deps=[trips])
def daily_service_performance():
    ...                       # gold

# + a ScheduleDefinition that
#   materializes the assets at
#   02:00 Asia/Bangkok
```

Airflow 3 TaskFlow API · same idea everywhere: steps, dependencies, schedule, retries, a UI

Open-source alternatives

Need	Used today	Open-source alternatives	Notes
Experiment tracking	MLflow Tracking	Aim, ClearML, DVC + DVCLive	DVC keeps experiments in Git
Model registry	MLflow Model Registry	ClearML, Kubeflow Model Registry	
Orchestration	Prefect	Apache Airflow, Dagster, Kestra, Luigi, Argo Workflows	Kestra flows are YAML; Argo runs on Kubernetes
ML pipelines on Kubernetes	–	Kubeflow Pipelines, Flyte	each step is a container
All-in-one ML framework	–	Metaflow, ZenML	Metaflow came from Netflix
Table versions	DuckLake snapshots	Apache Iceberg, Delta Lake, lakeFS	lakeFS adds Git-like branches

Weights & Biases and Comet are popular hosted, commercial trackers. Choose by where it runs (laptop, Kubernetes, cloud) and what it centers on: tasks or tables.

Wrap-up

Key ideas

- DuckLake versions data, MLflow versions experiments and models, Prefect versions pipeline runs
- One MLflow run per fit: params, metrics, model and the DuckLake snapshot_id
- Load models by alias; promote or roll back by moving it
- Prefect adds order, retries, logs and schedules; idempotent steps make retries safe
- Airflow, Dagster, DVC and Kubeflow apply the same ideas

Try it yourself

1. Log the random forest from Exercise 2 as a run; compare it in the UI
2. Write check_gold: fail the flow if September's rainy cancellation rate is not 11.11%
3. Log the rain threshold (1.0 mm) as a param; try 0.5 and 2.0
4. Promote only if MAE improves by at least 0.05 minutes
5. Backfill June–August with prefect deployment run